



비행기표는 언제 예매해야 가장 효과적일까?

kuggle 1차 프로젝트 3조

목차

01

주제 선정 이유

03

데이터 전처리

05

Test Data 예측

02

분석 개요

04

EDA / 모델링 및 튜닝

06

결론 도출

주제 선정 이유



- 항공권 가격 변동 → 많은 사람들이 궁금해하는 주제
- 데이터 분석으로 합리적 예매 시점 탐구
- 실생활에 직접 도움, 소비 결정에 기여

분석 개요

Airlines Flights Data

<https://www.kaggle.com/datasets/rohitgrewal/airlines-flights-data>

Columns

- | | |
|------------------|--------------------|
| - airline | - destination_city |
| - flight | - class |
| - source_city | - duration |
| - departure_time | - days_left |
| - stops | - price |
| - arrival_time | - num_stops |

데이터 전처리

```
import pandas as pd
import numpy as np
from tqdm.auto import tqdm
```

```
file_path = './airlines_flights_data.csv'
df_air = pd.read_csv(file_path, encoding='ascii')
```

컬럼명 통일: 소문자로 바꾸고, 공백은 밑줄(_)로 바꾸기

```
df_air.columns = [c.strip().lower().replace(' ', '_') for c in df_air.columns]
```

문자열 데이터 앞뒤 공백 제거

```
for col in df_air.select_dtypes(include=['object']).columns:
    df_air[col] = df_air[col].astype(str).str.strip()
```

숫자로 변환할 수 있는 컬럼 숫자형으로 변환

```
numeric_candidates = ['duration', 'days_left', 'price']
for col in numeric_candidates:
    if col in df_air.columns:
        df_air[col] = pd.to_numeric(df_air[col], errors='coerce')
```

CHAPTER 3

범주형 텍스트(예 : 출발 시간, 경유 등)를 일관된 형식으로 정리

```
cat_cols= ['airline', 'flight', 'source_city',  
           'departure_time', 'stops', 'arrival_time',  
           'destination_city', 'class']  
for col in [c for c in cat_cols if c in df_air.columns]:  
    df_air[col] = df_air[col].str.replace('-', '_').str.replace('__', '_')
```

완전히 동일한 중복 행 제거

```
before = len(df_air)  
df_air = df_air.drop_duplicates()  
after = len(df_air)
```

가격과 소요시간(duration)에서 명확한 이상치(IQR 기준) 처리
→ 상 · 하한선으로 자르기

```
for col in ['price', 'duration']:  
    if col in df_air.columns:  
        q1 = df_air[col].quantile(0.25)  
        q3 = df_air[col].quantile(0.75)  
        iqr = q3 - q1  
        lower = q1 - 1.5*iqr  
        upper = q3 + 1.5*iqr  
        df_air[col] = df_air[col].clip(lower=lower, upper=upper)
```

CHAPTER 3

결측치 처리: 비율이 아주 작은 경우는 해당 행 제거,
그 외는 모델 단계에서 처리하도록 둬

```
na_ratio = df_air.isna().mean()
cols_to_drop_na = [c for c in df_air.columns if na_ratio[c]
                    > 0 and na_ratio[c] < 0.02]
if len(cols_to_drop_na) > 0:
    df_air = df_air.dropna(subset=cols_to_drop_na)
```

정제된 데이터셋 저장

```
clean_path = 'airlines_flights_data_clean.csv'
df_air.to_csv(clean_path, index=False)
```

데이터 미리보기

```
print(df_air.head())
print(df_air.describe(include='all').transpose().head())
print(clean_path)
```

출력

	index	airline	flight	source_city	departure_time	stops	arrival_time	W
0	0	SpiceJet	SG_8709	Delhi	Evening	zero	Night	
1	1	SpiceJet	SG_8157	Delhi	Early_Morning	zero	Morning	
2	2	AirAsia	I5_784	Delhi	Early_Morning	zero	Early_Morning	
3	3	Vistara	UK_995	Delhi	Morning	zero	Afternoon	
4	4	Vistara	UK_963	Delhi	Morning	zero	Morning	

	destination_city	class	duration	days_left	price	num_stops
0	Mumbai	Economy	2.17	1	5953	0
1	Mumbai	Economy	2.33	1	5953	0
2	Mumbai	Economy	2.17	1	5956	0
3	Mumbai	Economy	2.25	1	5955	0
4	Mumbai	Economy	2.33	1	5955	0

	count	unique	top	freq	mean	std	min	W
index	300153.0	NaN	NaN	NaN	150076.0	86646.852011	0.0	
airline	300153	6	Vistara	127859	NaN	NaN	NaN	
flight	300153	1561	UK_706	3235	NaN	NaN	NaN	
source_city	300153	6	Delhi	61343	NaN	NaN	NaN	
departure_time	300153	6	Morning	71146	NaN	NaN	NaN	

	25%	50%	75%	max
index	75038.0	150076.0	225114.0	300152.0
airline	NaN	NaN	NaN	NaN
flight	NaN	NaN	NaN	NaN
source_city	NaN	NaN	NaN	NaN
departure_time	NaN	NaN	NaN	NaN

airlines_flights_data_clean.csv

EDA / 모델링 및 튜닝

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 업로드된 CSV 파일 읽기 (인코딩 지정)
file_path = './airlines_flights_data_clean.csv'
df_flights = pd.read_csv(file_path, encoding='ascii')
```

데이터 크기(shape)와 앞부분 미리보기

```
print(df_flights.shape)
print(df_flights.head())
```

숫자형 컬럼 요약 통계

```
print(df_flights.describe(include=[np.number]))
```

출력

```
(300153, 13)
   index  airline  flight  source_city  departure_time  stops  arrival_time  ₩
0      0  SpiceJet  SG_8709      Delhi      Evening    zero      Night
1      1  SpiceJet  SG_8157      Delhi  Early_Morning    zero      Morning
2      2  AirAsia  I5_764      Delhi  Early_Morning    zero  Early_Morning
3      3  Vistara  UK_995      Delhi      Morning    zero      Afternoon
4      4  Vistara  UK_963      Delhi      Morning    zero      Morning
```

```
   destination_city  class  duration  days_left  price  num_stops
0          Mumbai  Economy      2.17         1   5953          0
1          Mumbai  Economy      2.33         1   5953          0
2          Mumbai  Economy      2.17         1   5956          0
3          Mumbai  Economy      2.25         1   5955          0
4          Mumbai  Economy      2.33         1   5955          0
```

```
count      300153.000000  300153.000000  300153.000000  300153.000000
mean       150076.000000    12.199951    26.004751   20887.402112
std        86646.852011     7.131314    13.561004   22689.507652
min           0.000000     0.830000     1.000000    1105.000000
25%        75038.000000     6.830000    15.000000    4783.000000
50%       150076.000000    11.250000    26.000000    7425.000000
75%       225114.000000    16.170000    38.000000   42521.000000
max       300152.000000    30.180000    49.000000   99128.000000
```

```
count      300153.000000
mean         0.924312
std         0.398106
min           0.000000
25%          1.000000
50%          1.000000
75%          1.000000
max           2.000000
```


범주형 컬럼 요약

```
categorical_cols = df_flights.select_dtypes(include=['object']).columns.tolist()
print(categorical_cols)
print(df_flights[categorical_cols].nunique())
```

출력

```
['airline', 'flight', 'source_city', 'departure_time', 'stops', 'arrival_time', 'destination_city', 'class']
airline          6
flight          1561
source_city      6
departure_time   6
stops           3
arrival_time     6
destination_city 6
class           2
dtype: int64
```

CHAPTER 4

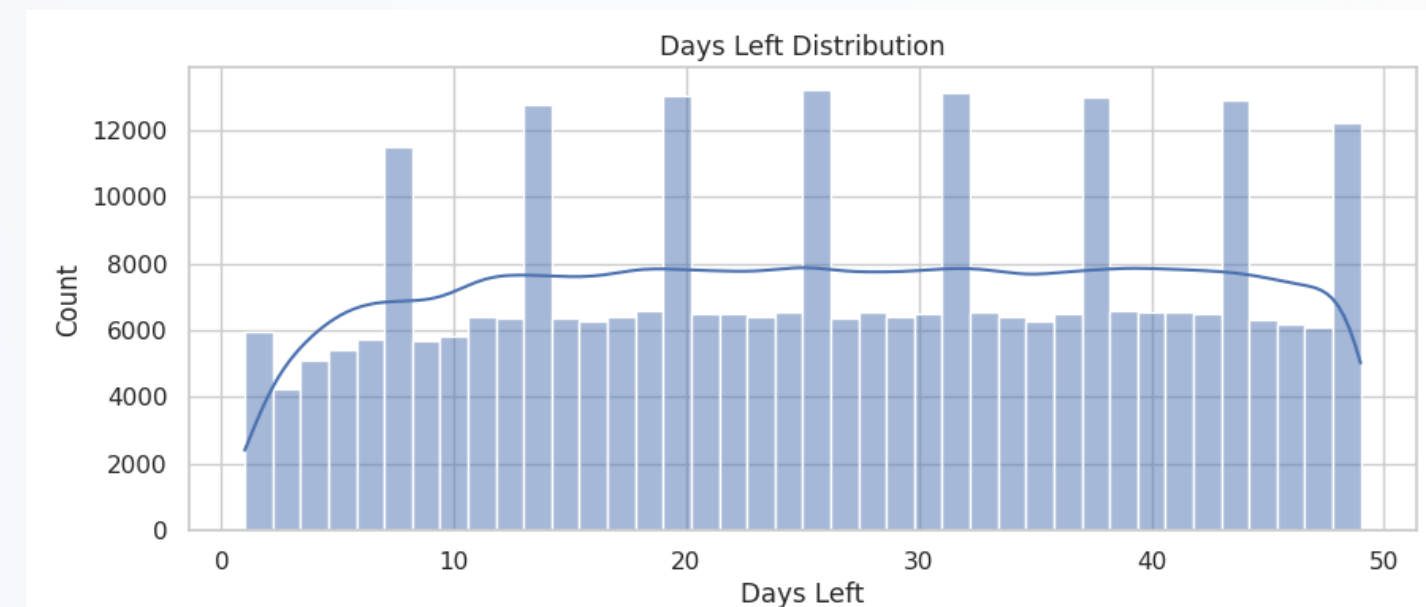
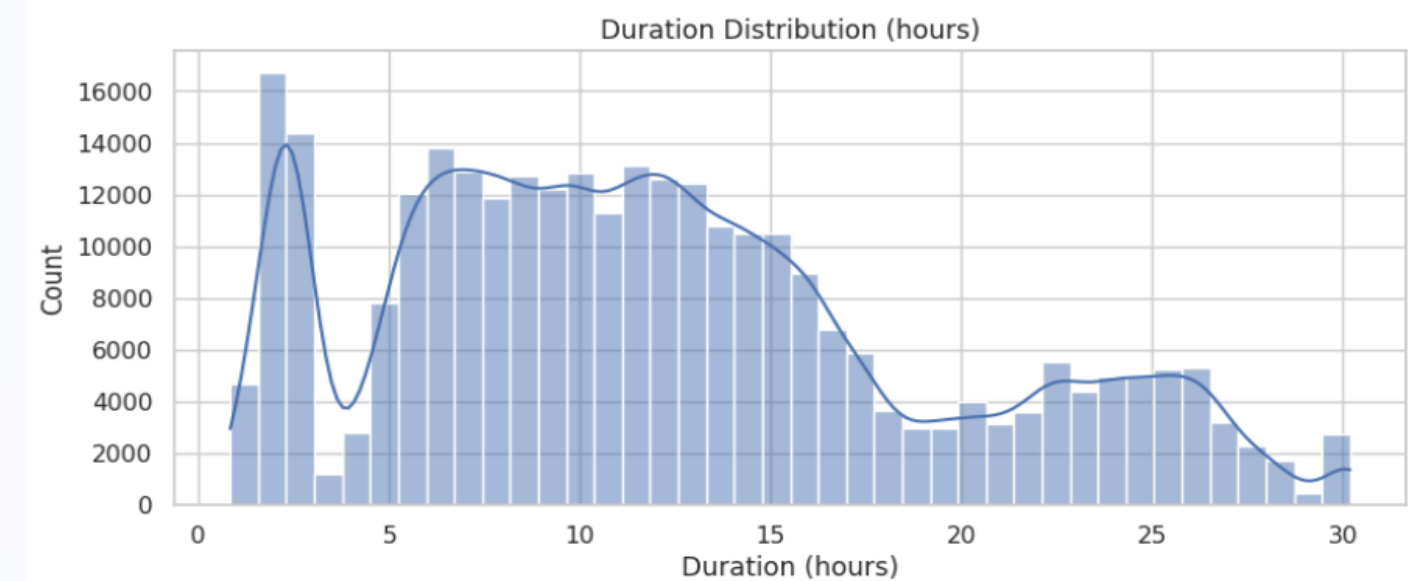
주요 숫자형 변수들의 분포 시각화

```
sns.set(style='whitegrid')
plt.figure(figsize=(10,4))
sns.histplot(df_flights['price'], bins=40, kde=True)
plt.title('Price Distribution')
plt.xlabel('Price')
plt.ylabel('Count')
plt.show()
```

```
plt.figure(figsize=(10,4))
sns.histplot(df_flights['duration'], bins=40, kde=True)
plt.title('Duration Distribution (hours)')
plt.xlabel('Duration (hours)')
plt.ylabel('Count')
plt.show()
```

```
plt.figure(figsize=(10,4))
sns.histplot(df_flights['days_left'], bins=40, kde=True)
plt.title('Days Left Distribution')
plt.xlabel('Days Left')
plt.ylabel('Count')
plt.show()
```

출력



모델 학습

1. RandomForest

2. GradientBoost

3. XGBoost

3가지 모델 선정 이유

→ 성능과 해석력의 균형을 고려한 대표적인 모델들이기 때문

RandomForest

- 전체흐름: 데이터를 전처리(샘플링·인코딩)
학습/테스트 분할 랜덤포레스트 모델 학습
RMSE·R² 성능 지표 출력

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd

# 1. 데이터 준비
X = df.drop(columns=['price', 'index', 'flight'])
y = df['price']
X = pd.get_dummies(X, drop_first=True)

# 속도 단축: 데이터 10% 샘플링 (원하면 frac=1.0으로 전체 사용)
X, y = X.sample(frac=0.1, random_state=42), y.sample(frac=0.1,
                                                       random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2. 랜덤포레스트 모델 정의 & 학습

```
rf = RandomForestRegressor(
    random_state=42,
    n_estimators=100,
    n_jobs=-1
)
rf.fit(X_train, y_train)
```

3. 예측

```
y_pred = rf.predict(X_test)
```

4. 성능 평가

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

```
print("랜덤포레스트 성능 지표")
print("RMSE:", round(rmse, 2))
print("R2 :", round(r2, 4))
```

랜덤포레스트 성능 지표

RMSE: 3584.73

R² : 0.9748

GradientBoost

- 전체흐름: 데이터를 전처리(샘플링·인코딩)
학습/테스트 분할 그레디언트부스트 모델 학습
MAE·R² 성능 지표 출력

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.inspection import PartialDependenceDisplay

# 1) 데이터 로드 (데이터 가져오기)
df_flights = pd.read_csv('./airlines_flights_data_clean.csv', encoding='utf-8')

# 2) 특성/타겟 분리 (예측에 사용할 입력 변수들 features로 넣기)

features = ['days_left', 'duration', 'airline', 'source_city',
            'destination_city', 'departure_time', 'arrival_time',
            'stops', 'class']
X = df_flights[features].copy()
y = df_flights['price'].astype(float)

# 3) 학습/검증 분리
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

```
# 4) 전처리 파이프라인
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
preprocess = ColumnTransformer([
    ('num', 'passthrough', ['days_left', 'duration']),
    ('cat', encoder, ['airline', 'source_city', 'destination_city',
                    'departure_time', 'arrival_time', 'stops', 'class'])
])
```

```
# 5) 모델 구성/학습 - 모델+파이프라인 학습
gbr = GradientBoostingRegressor(random_state=42)
model = Pipeline(steps=[('preprocess', preprocess), ('model', gbr)])
model.fit(X_train, y_train)
```

```
# 6) 성능평가
preds = model.predict(X_test)
mae = mean_absolute_error(y_test, preds)
r2 = r2_score(y_test, preds)
print(mae)
print(r2)
```

그레디언트부스트 성능 지표

MAE: 2936.91

R² : 0.9520595156610072

XGBoost

- 전체흐름: 데이터를 전처리(샘플링·인코딩)
학습/테스트 분할 XGBoost 모델 학습
RMSE·R² 성능 지표 출력

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import pandas as pd
from xgboost import XGBRegressor

# 1. 데이터 준비
X = df.drop(columns=['price', 'index', 'flight'])
y = df['price']
X = pd.get_dummies(X, drop_first=True)

# 속도 단축: 데이터 10% 샘플링 (원하면 frac=1.0으로 전체 사용)
X, y = X.sample(frac=0.1, random_state=42), y.sample(frac=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. XGBoost 모델 정의 & 학습
xgb = XGBRegressor(
    random_state=42,
    n_estimators=300, # 트리 개수 (조절 가능)
    learning_rate=0.1, # 학습률
    max_depth=6, # 트리 깊이
    n_jobs=-1,
    subsample=0.8,
    colsample_bytree=0.8
)
xgb.fit(X_train, y_train)
```

```
# 3. 예측
y_pred = xgb.predict(X_test)

# 4. 성능 평가
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("XGBoost 성능 지표")
print("RMSE:", round(rmse, 2))
print("R² :", round(r2, 4))
```

```
RMSE: 3384.9
R² : 0.9778
```

CHAPTER 5

전처리 및 XGBoostRegressor 모델 파이프라인 정의

```
prep = ColumnTransformer([
    # 범주형 변수
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
     cat_cols),
    # 수치형 변수
    ('num', 'passthrough', num_cols)
])
```

```
# XGBoost 회귀 모델 설정 (히스토그램 기반 트리 사용)
xgb = XGBRegressor(random_state=42, tree_method='hist')
```

```
# 전처리와 모델을 하나의 파이프라인으로 결합
pipe = Pipeline(steps=[('prep', prep), ('model', xgb)])
```

```
# 하이퍼파라미터 탐색을 위한 범위 설정 (랜덤 서치용)
param_dist = {
    'model__n_estimators': [200, 400, 600, 800, 1000],
    'model__max_depth': [3, 4, 5, 6, 7, 8],
    'model__learning_rate': [0.01, 0.05, 0.1, 0.2],
    'model__subsample': [0.6, 0.8, 1.0],
    'model__colsample_bytree': [0.6, 0.8, 1.0],
    'model__gamma': [0, 0.1, 0.3, 0.5]
}
```

RandomizeSearchCV를 이용한 모델 튜닝 설정

```
random_search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_dist,
    n_iter=20,
    scoring='neg_mean_absolute_error',
    cv=3,
    verbose=2,
    random_state=42,
    n_jobs=-1
)
```

```
# 모델 학습 및 최적 하이퍼파라미터 탐색 수행
random_search.fit(X_train, y_train)
```

```
# 최적 파라미터 출력
print("Best Parameters:\n", random_search.best_params_)
```

모델 성능 비교

1. RandomForest : $R^2 = 0.9748$

2. GradientBoost : $R^2 = 0.9520$

3. XGBoost : $R^2 = 0.9845$

⇒ **XGBoost**가 R^2 값 기준으로 가장 높은 성능을 보여 주었음.
따라서 예측 정확도 면에서 **가장 우수함**.

Test Data 예측

전체 흐름: 튜닝된 최적 모델을 테스트셋에 적용 → 평가지표
RMSE와 R^2 계산 → 시각화로 결과 확인

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
```

```
#튜닝된 최적 모델 가져오기
best_model = random_search.best_estimator_ # 전처리 + XGB가 다 들어 있는 파이프 라인

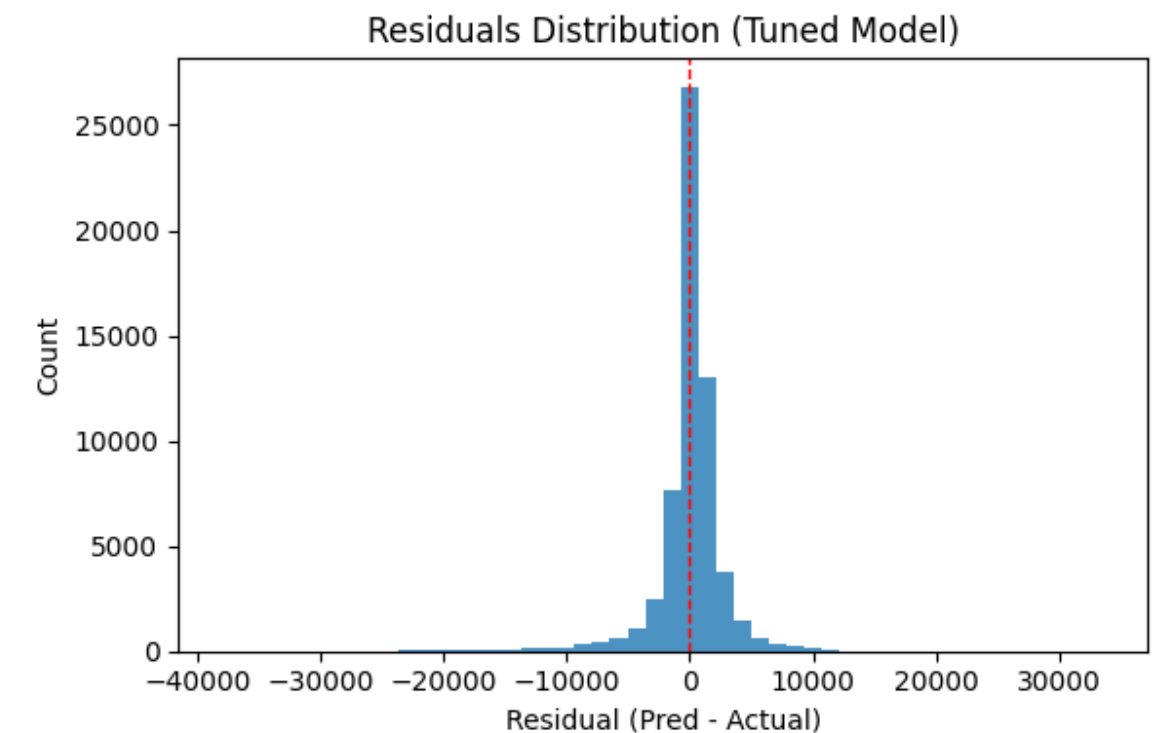
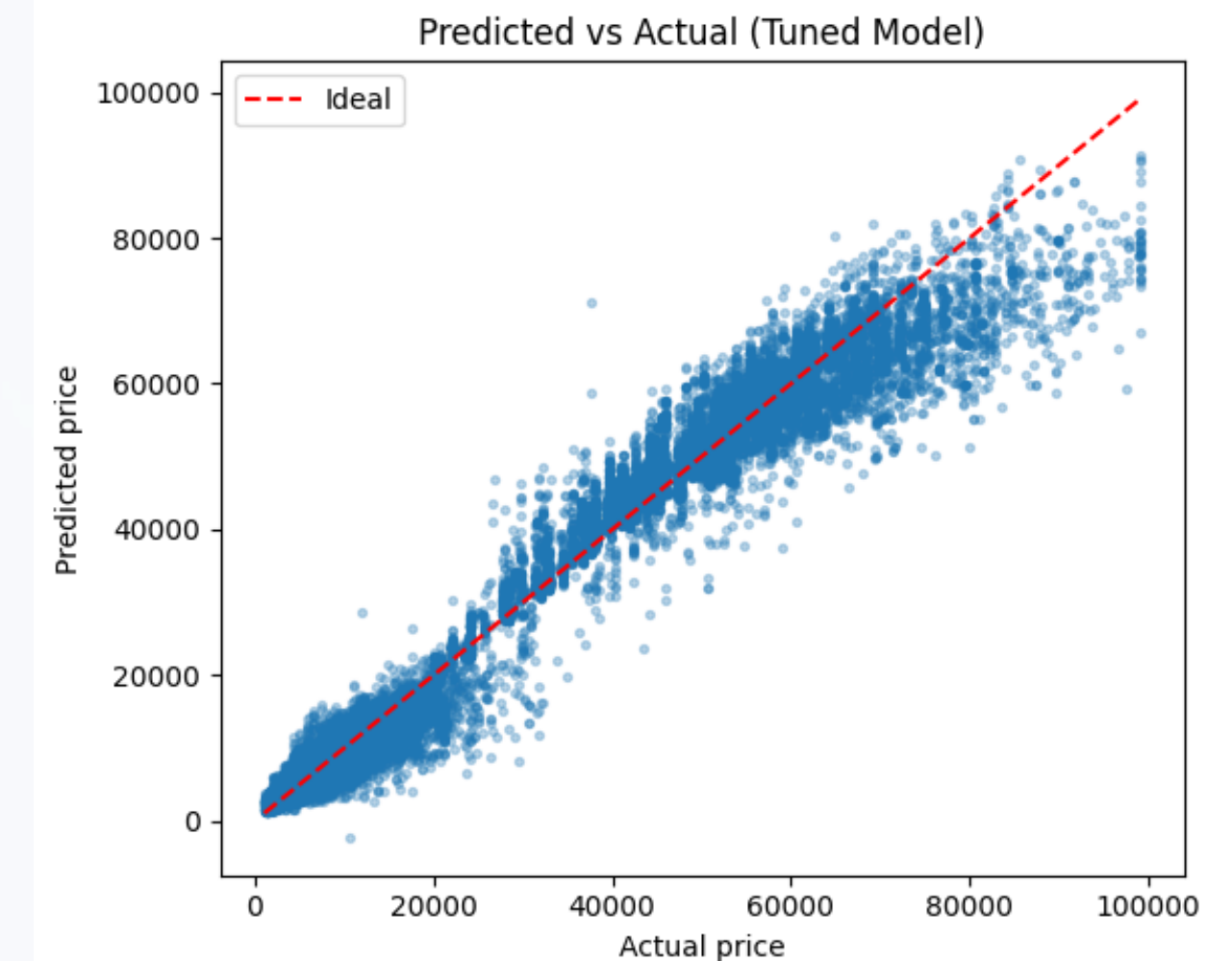
#테스트셋 예측
preds_best = best_model.predict(X_test)

#NaN/Inf 마스킹
mask = np.isfinite(y_test) & np.isfinite(preds_best)

#평가지표 RMSE와  $R^2$  계산
rmse_best = float(np.sqrt(mean_squared_error(y_test[mask], preds_best[mask])))
r2_best = float(r2_score(y_test[mask], preds_best[mask]))

print("최적 모델 RMSE:", round(rmse_best, 2))
print("최적 모델  $R^2$  :", round(r2_best, 4))
```

최적 모델 RMSE: 2825.71
최적 모델 R^2 : 0.9845



CHAPTER 5

최저점(최저가 예약일) 찾기

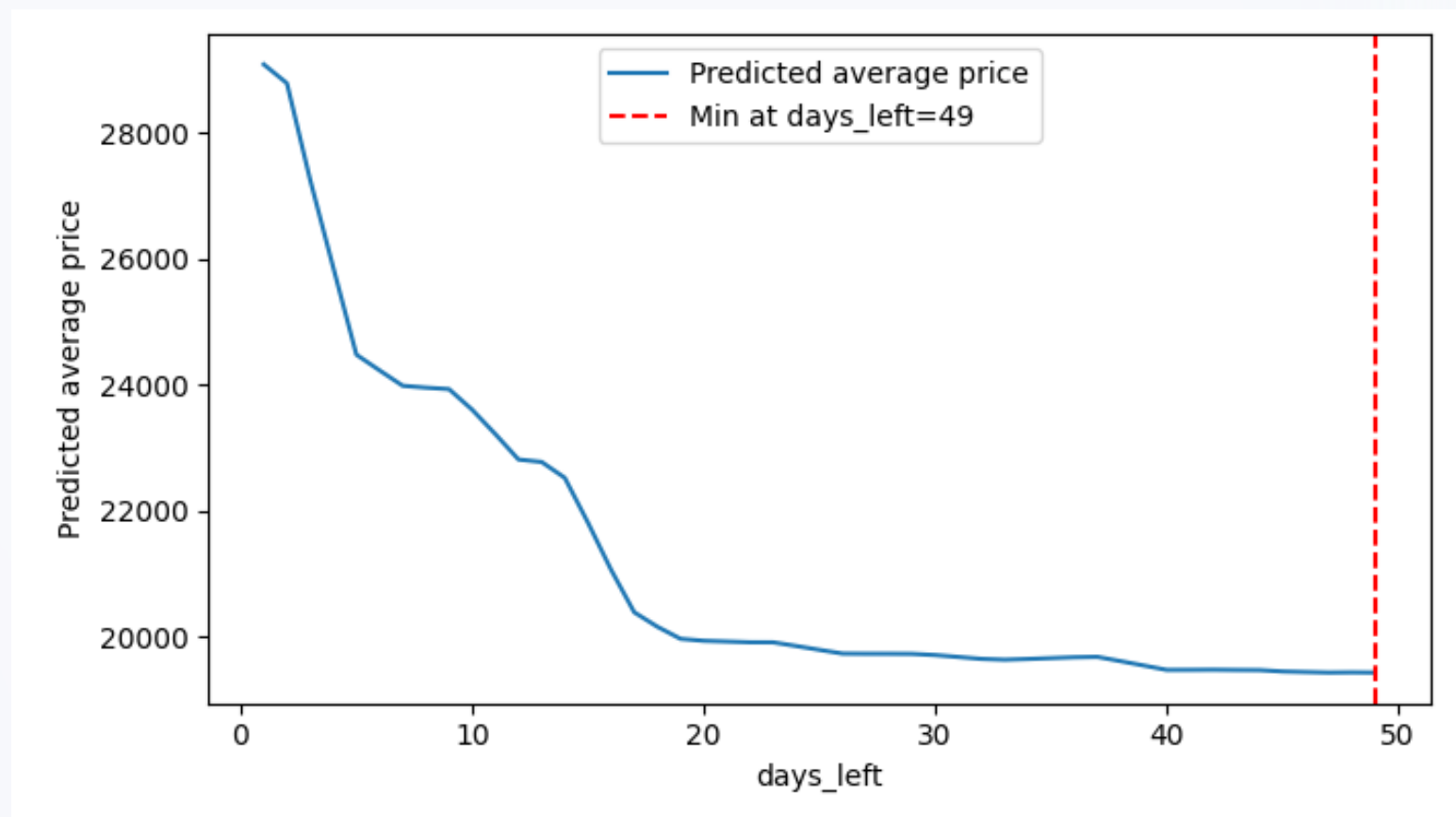
```
min_idx = int(curve['avg_price_smooth'].idxmin())
opt_days = int(curve.loc[min_idx, 'days_left'])
opt_price = float(curve.loc[min_idx, 'avg_price_smooth'])
```

시각화

```
plt.figure(figsize=(7,4))
plt.plot(curve['days_left'], curve['avg_price_smooth'],
         label='Predicted average price')
plt.axvline(opt_days, color='red', linestyle='--',
            label='Min at days_left=' + str(opt_days))
plt.xlabel('days_left')
plt.ylabel('Predicted average price')
plt.legend()
plt.tight_layout()
plt.show()
```

```
print(opt_days)
print(round(opt_price, 2))
print('Computed price vs. days_left curve and optimal booking day')
```

출력



49

19429.86

Computed price vs. days_left curve and optimal booking day

CHAPTER 5

전체 흐름: 모델로 예측 → 평가지표 RMSE와 R^2 계산
→ 시각화로 결과 확인

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
```

```
#이미 학습된 pipe_clean으로 테스트셋 예측에 대한 예측 가격 얻기
preds = pipe_clean.predict(X_test)
```

```
# NaN/Inf 마스킹
mask = np.isfinite(y_test) & np.isfinite(preds)
```

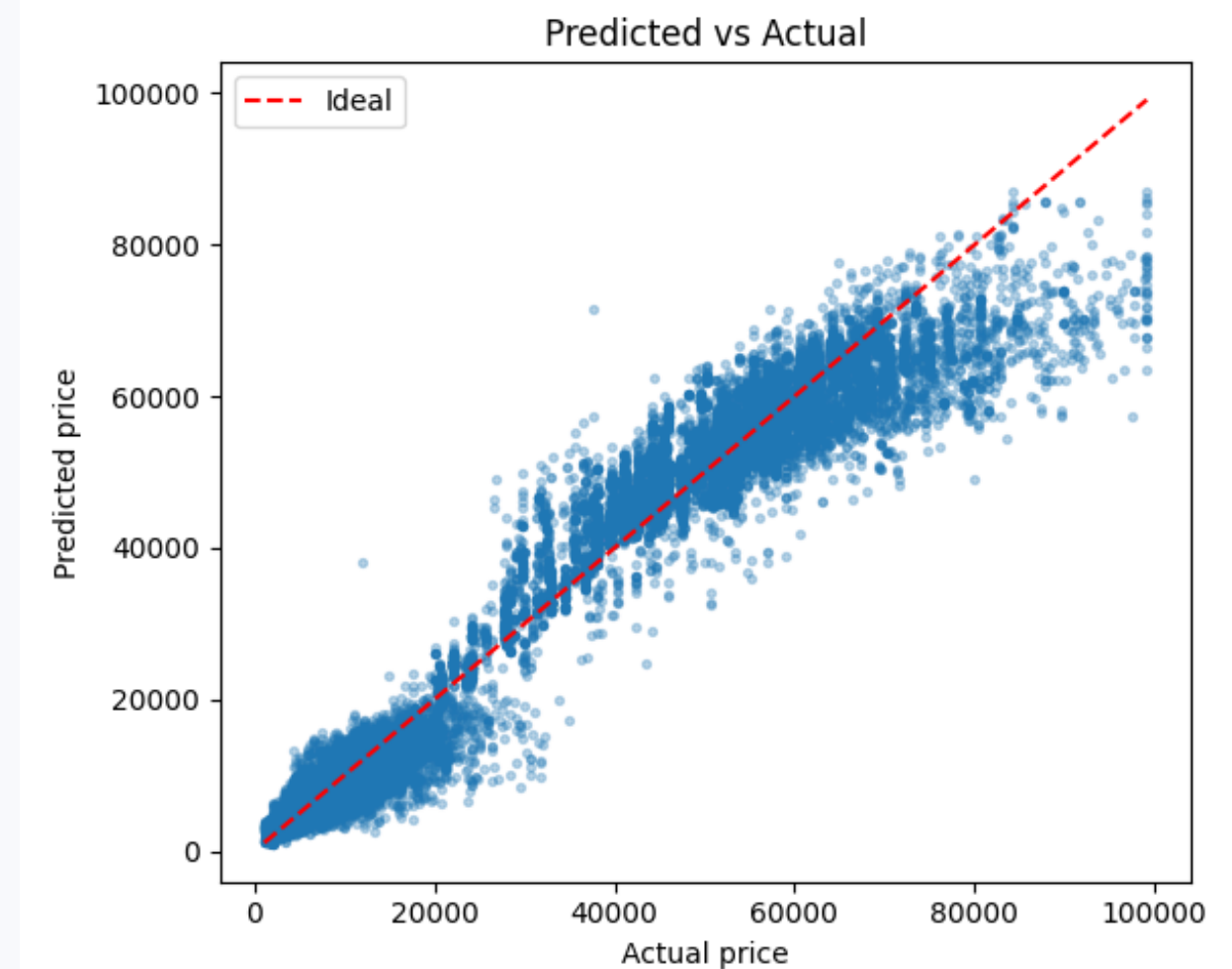
```
#평가지표 RMSE와  $R^2$  계산
rmse = float(np.sqrt(mean_squared_error(y_test[mask], preds[mask])))
r2 = float(r2_score(y_test[mask], preds[mask]))
```

```
print("RMSE:", round(rmse, 2))
print("R2 :", round(r2, 4))
```

```
RMSE: 3384.9
R2 : 0.9778
```

시각화 1: 실제값 vs 예측값 산점도

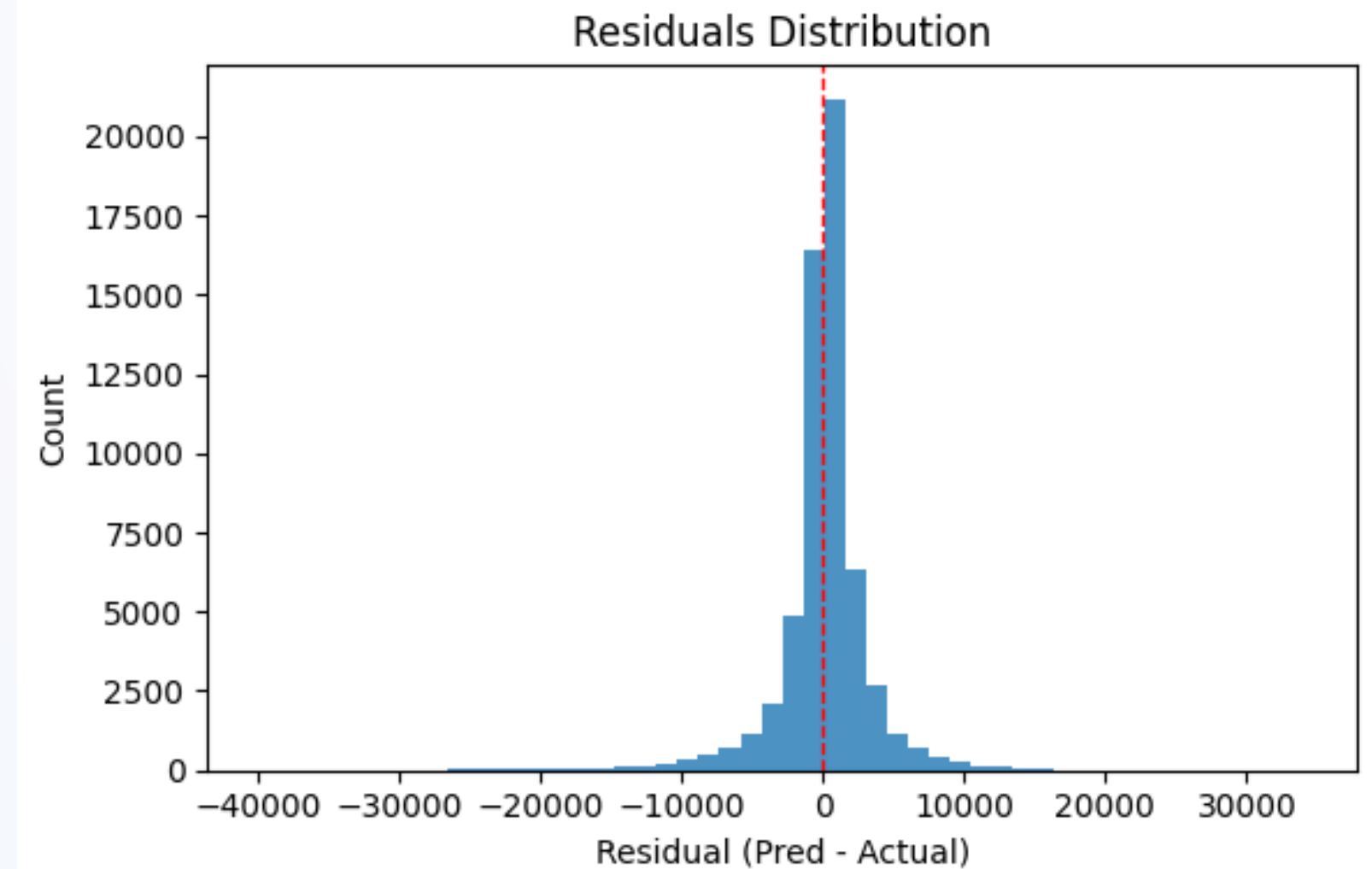
```
plt.figure(figsize=(6,5))
plt.scatter(y_test[mask], preds[mask], s=8, alpha=0.3)
plt.plot([y_test[mask].min(), y_test[mask].max()],
         [y_test[mask].min(), y_test[mask].max()],
         'r--', lw=1.5, label='Ideal') # 이상적 예측선
plt.xlabel('Actual price')
plt.ylabel('Predicted price')
plt.title('Predicted vs Actual')
plt.legend()
plt.tight_layout()
plt.show()
```



CHAPTER 5

시각화 2: 잔차 분포 히스토그램

```
resid = preds[mask] - y_test[mask]
plt.figure(figsize=(6,4))
plt.hist(resid, bins=50, alpha=0.8)
plt.axvline(0, color='red', linestyle='--', linewidth=1)
plt.xlabel('Residual (Pred - Actual)')
plt.ylabel('Count')
plt.title('Residuals Distribution')
plt.tight_layout()
plt.show()
```



결론 도출

데이터 분석 결과:

가장 합리적인 시점

→ 출발일 기준 49일 전에 예약

그때의 가격

→ 19,429 루피 (= 30만 6408원)



질의 응답 Q&A